

Natural Language Processing

2. Deep Learning Basics

CBNU

School of Computer Science
Emeritus Prof. Lee, Jae Sung

Table of Contents

1. Basic principle of neural networks
2. Perceptron and backpropagation
3. Recurrent Neural Networks (RNN)
4. Attention mechanism
5. Transformer architecture
6. Transformer-based language models
7. Applications such as BERT and GPT

Deep Learning

❖ Deep Learning

- Deep Learning is a machine learning technique based on neural networks with multiple layers.
- It automatically learns representations from data.

❖ Examples:

- Image recognition
- Speech recognition
- Natural language processing
- Autonomous driving

Machine Learning vs. Deep Learning

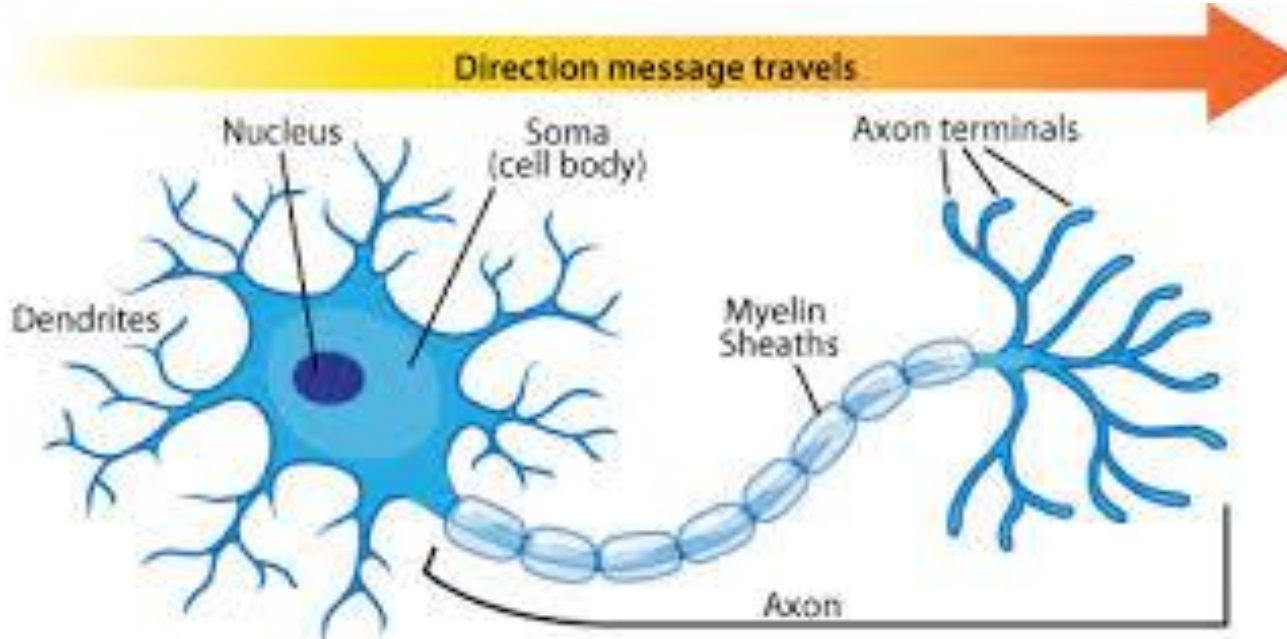
❖ Machine Learning

- Requires manual feature engineering
- Uses models such as SVM, decision trees

❖ Deep Learning

- Automatically learns features
- Uses multi-layer neural networks

Biological Neural Network (Neuron)



<https://askabiologist.asu.edu/neuron-anatomy>

Artificial Neural Networks

- ❖ Artificial Neural Networks are inspired by biological neurons.

$$y = f(w_1x_1 + w_2x_2 + \cdots + w_nx_n + b)$$

where

w_i :weights

b : bias

f : activation function

- ❖ Common activation functions:

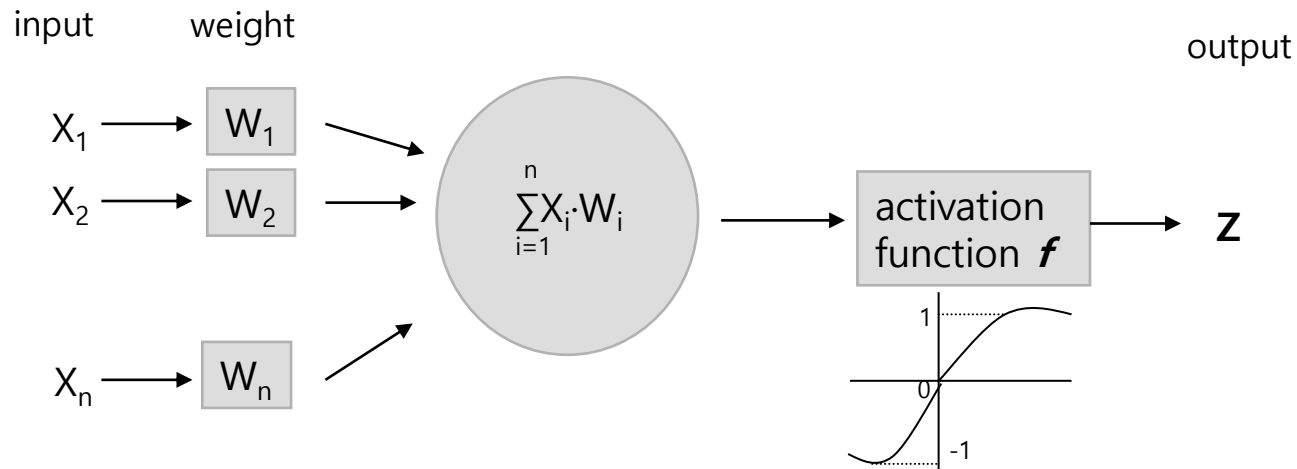
- Sigmoid
- Tanh
- ReLU
- Softmax

- ❖ ReLU is widely used in deep learning.

- $ReLU(x) = \max(0, x)$

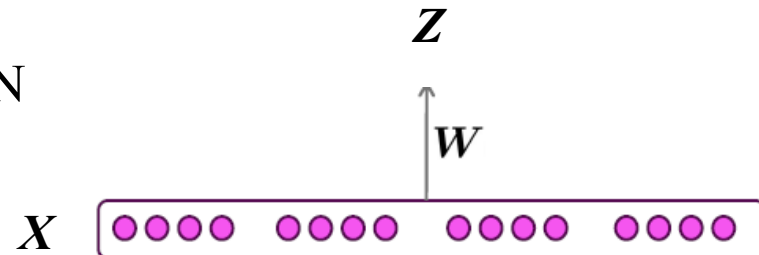
Diagrams of Artificial Neural Net

❖ Diagram of artificial neural net



❖ Formula and simple notation of ANN

$$Z = f(X \cdot W + b)$$



Perceptron Model (1960-1990's)

❖ Single Layer Perceptron

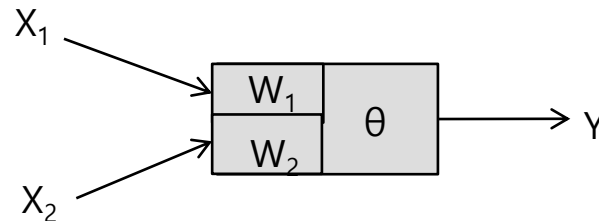
- The perceptron is the **simplest neural network model**.
- It performs **binary classification**.

$$y = \text{sign}(w \cdot x + b)$$

- Inputs are linearly combined and passed through a threshold.

❖ Example

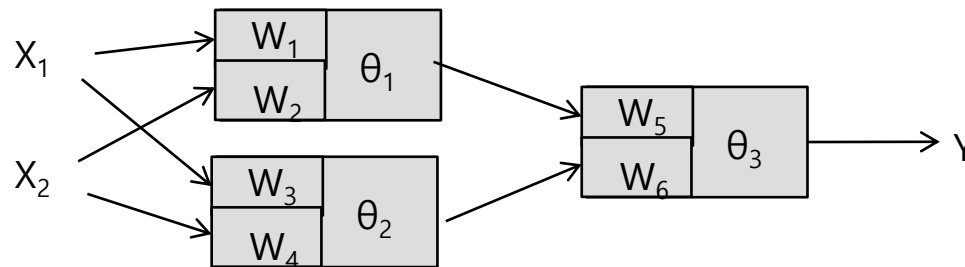
- Input: X_1, X_2 Weight: W_1, W_2 Output: Y Threshold: θ
- if $X_1 * W_1 + X_2 * W_2 > \theta$, $Y=1$, else $Y=0$
- Possible implementation: AND, OR function
- Impossible implementation: XOR function



Perceptron Model (1960-1990's)

❖ Multi Layer Perceptron

- Neural networks with hidden layers can learn complex patterns.
- Structure:
 - Input layer
 - Hidden layer(s)
 - Output layer
- This architecture enables **nonlinear decision boundaries**.



Training Neural Networks

❖ Training neural networks involves minimizing a **loss function**.

❖ Example loss functions:

- Mean Squared Error
- Cross Entropy

❖ Goal:

$$\min L(\theta)$$

where θ are model parameters.

Gradient Descent

- ❖ Gradient Descent updates parameters using the gradient of the loss.

$$w = w - \eta \nabla L$$

where

η = learning rate

- ❖ This allows the model to gradually reduce errors.

Backpropagation

- ❖ Backpropagation computes gradients efficiently using the **chain rule**.
- ❖ Steps:
 - 1 Forward pass
 - 2 Compute loss
 - 3 Backward pass (gradient calculation)
 - 4 Update weights
- ❖ Backpropagation enables training deep neural networks.

Backpropagation Example

❖ For a neuron:

$$y = f(wx + b)$$

❖ Loss:

$$L = (y - t)^2$$

❖ Using the chain rule:

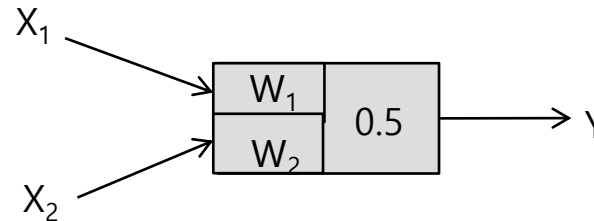
$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial w}$$

❖ Gradients propagate from output to input layers.

Perceptron Training Example (1960-1990's)

❖ Training OR operator

- Input: x_1, x_2
- Target value: T
- Termination condition:
output correct answers
for all input ($d=0$ for all)



Step	x1	x2	T	초기값(o)		Y	d	변경값(n)	
				W1	W2			W1	W2
1	0	0	0	0.1	0.3	0	0	0.1	0.3

$$W_n = W_o + a \cdot d \cdot X$$

$$d = T - Y$$

W_n : new weight

W_o : old(current) weight

a : learning rate (default 0.2)

θ : threshold (default 0.5)

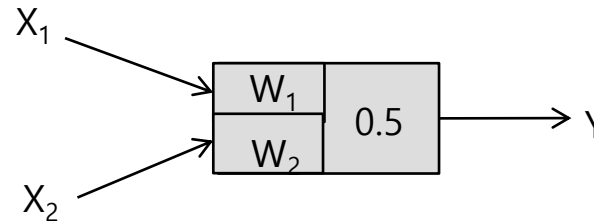
$$Y = (0.1 \cdot 0 + 0.3 \cdot 0) > 0.5 = 0 \quad W1 = 0.1 + 0.2 \cdot 0 \cdot 0 = 0.1$$

$$d = 0 - 0 = 0 \quad W2 = 0.3 + 0.2 \cdot 0 \cdot 0 = 0.3$$

Perceptron Training Example (1960-1990's)

❖ Training OR operator

- Input: x_1, x_2
- Target value: T
- Termination condition: output correct answers for all input ($d=0$ for all)



Step	x1	x2	T	초기값(o)		Y	d	변경값(n)	
				W1	W2			W1	W2
1	0	0	0	0.1	0.3	0	0	0.1	0.3
	0	1	1	0.1	0.3	0	1	0.1	0.5
	1	0	1	0.1	0.5	0	1	0.3	0.5
	1	1	1	0.3	0.5	1	0	0.3	0.5

$$W_n = W_o + a \cdot d \cdot X$$

$$d = T - Y$$

W_n : new weight

W_o : old(current) weight

a : learning rate (default 0.2)

θ : threshold (default 0.5)

$$Y = (0.1 \cdot 0 + 0.3 \cdot 0) > 0.5 = 0 \quad W1 = 0.1 + 0.2 \cdot 0 \cdot 0 = 0.1$$

$$d = 0 - 0 = 0 \quad W2 = 0.3 + 0.2 \cdot 0 \cdot 0 = 0.3$$

$$Y = (0.1 \cdot 0 + 0.3 \cdot 1) > 0.5 = 0 \quad W1 = 0.1 + 0.2 \cdot 1 \cdot 0 = 0.1$$

$$d = 1 - 0 = 1 \quad W2 = 0.3 + 0.2 \cdot 1 \cdot 1 = 0.5$$

$$Y = (0.1 \cdot 1 + 0.5 \cdot 0) > 0.5 = 0 \quad W1 = 0.1 + 0.2 \cdot 1 \cdot 1 = 0.3$$

$$d = 1 - 0 = 1 \quad W2 = 0.5 + 0.2 \cdot 1 \cdot 0 = 0.5$$

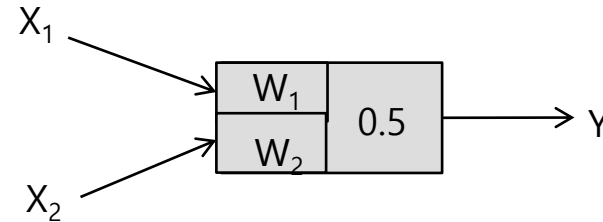
$$Y = (0.3 \cdot 1 + 0.5 \cdot 1) > 0.5 = 1 \quad W1 = 0.3 + 0.2 \cdot 0 \cdot 1 = 0.3$$

$$d = 1 - 1 = 0 \quad W2 = 0.5 + 0.2 \cdot 0 \cdot 1 = 0.5$$

Perceptron Training Example (1960-1990's)

❖ Training OR operator

- Input: x_1, x_2
- Target value: T
- Termination condition:
output correct answers
for all input ($d=0$ for all)



Step	x1	x2	T	초기값(o)		Y	d	변경값(n)	
				W1	W2			W1	W2
1	0	0	0	0.1	0.3	0	0	0.1	0.3
	0	1	1	0.1	0.3	0	1	0.1	0.5
	1	0	1	0.1	0.5	0	1	0.3	0.5
	1	1	1	0.3	0.5	1	0	0.3	0.5
2	0	0	0	0.3	0.5	0	0	0.3	0.5
	0	1	1	0.3	0.5	0	1	0.3	0.7
	1	0	1	0.3	0.7	0	1	0.5	0.7
	1	1	1	0.5	0.7	1	0	0.5	0.7
3	0	0	0	0.5	0.7	0	0	0.5	0.7
	0	1	1	0.5	0.7	1	0	0.5	0.7
	1	0	1	0.5	0.7	0	1	0.7	0.7
	1	1	1	0.7	0.7	1	0	0.7	0.7
4	0	0	0	0.7	0.7	0	0	0.7	0.7
	0	1	1	0.7	0.7	1	0	0.7	0.7
	1	0	1	0.7	0.7	1	0	0.7	0.7
	1	1	1	0.7	0.7	1	0	0.7	0.7

$$W_n = W_o + a \cdot d \cdot X$$

$$d = T - Y$$

W_n : new weight

W_o : old(current) weight

a : learning rate (default 0.2)

θ : threshold (default 0.5)

Modern Method for Perceptron Training

❖ Neural Model

- $Y = (w_1x_1 + w_2x_2) > \theta \Rightarrow$

$$z = w \cdot x + b$$

$$y = \text{sign}(z)$$

❖ Loss Function

- Use loss function instead of the difference

- $d = T - Y \Rightarrow$

$$L = \frac{1}{2}(t - y)^2$$

❖ Gradient Descent Update

- $W_n = W_o + \alpha dx \Rightarrow$

$$w = w - \eta \frac{\partial L}{\partial w}$$

❖ Gradient to Perceptron

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial w} = -(t - y)x \Rightarrow$$

$$\begin{aligned} w &= w - \eta \frac{\partial L}{\partial w} \\ &= w + \eta(t - y)x \\ &= w + \alpha dx \end{aligned}$$

Sequential Data Tasks

- ❖ Many tasks involve **sequential data**.
- ❖ Examples:
 - Text
 - Speech
 - Time series
- ❖ Standard neural networks cannot easily model sequence dependencies.
- ❖ RNNs are used for:
 - Language modeling
 - Machine translation
 - Speech recognition
 - Text generation

Recurrent Neural Network(RNN)

❖ RNNs are designed for sequential data.

- They maintain a **hidden state** that captures previous information.

$$h_t = f(Wx_t + Uh_{t-1})$$

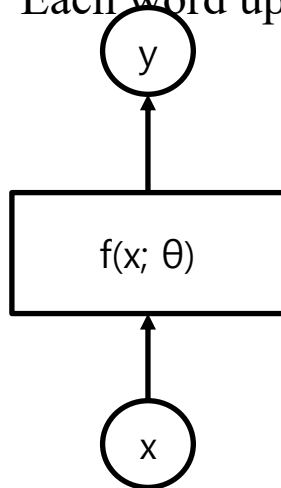
- The hidden state acts as memory.

❖ RNN processes sequences step by step.

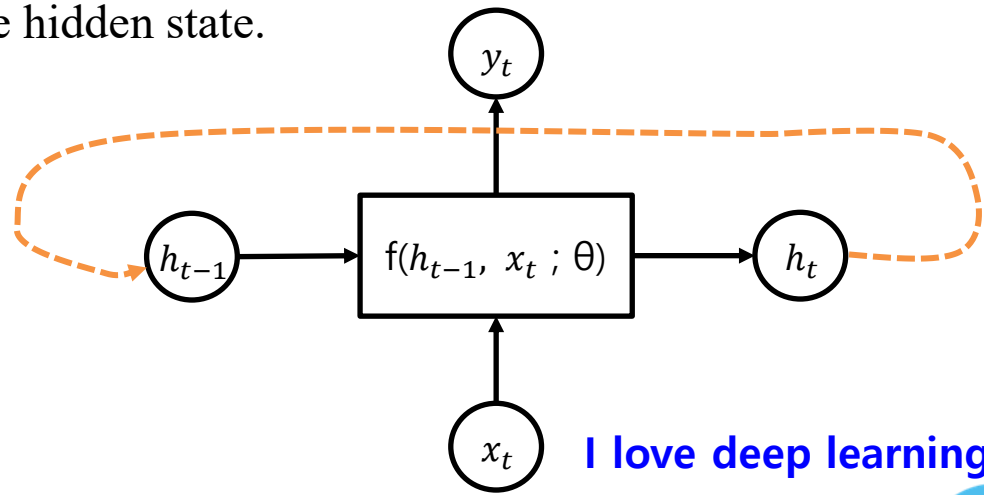
- Example sentence:

"I love deep learning"

- Each word updates the hidden state.



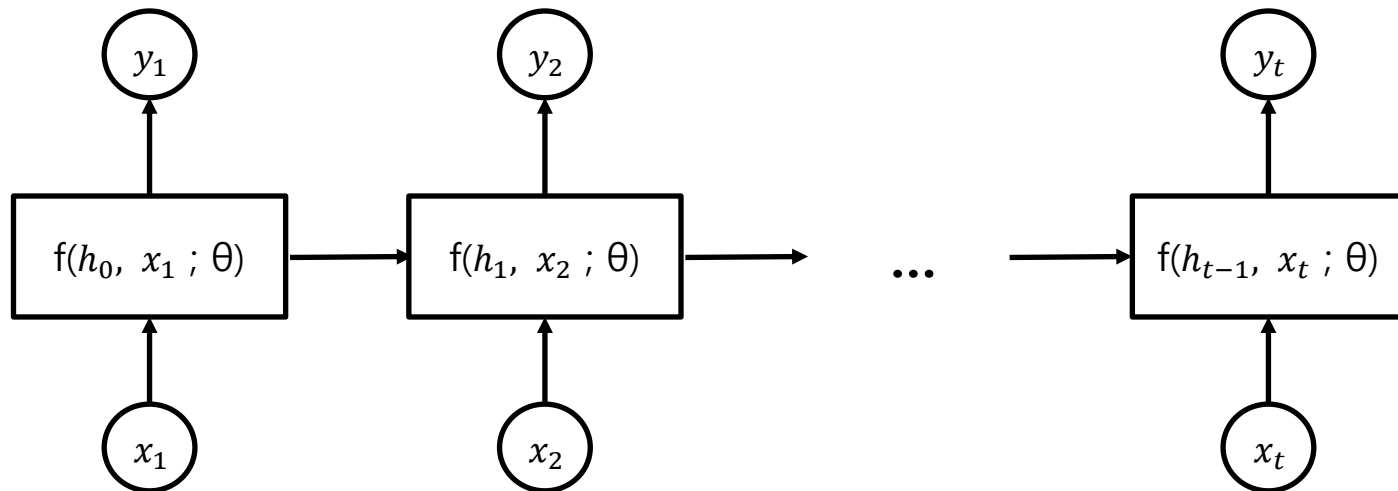
Traditional neural networks (feedforward networks) takes x as input and return y



RNNs take x_t and h_{t-1} as input and return y_t and h_t

Recurrent Neural Network(RNN)

- ❖ RNNs suffer from:
 - Vanishing gradient
 - Difficulty capturing long-range dependencies
 - Sequential computation (slow)
- ❖ These problems motivated new architectures.



Recurrent Neural Network(RNN)

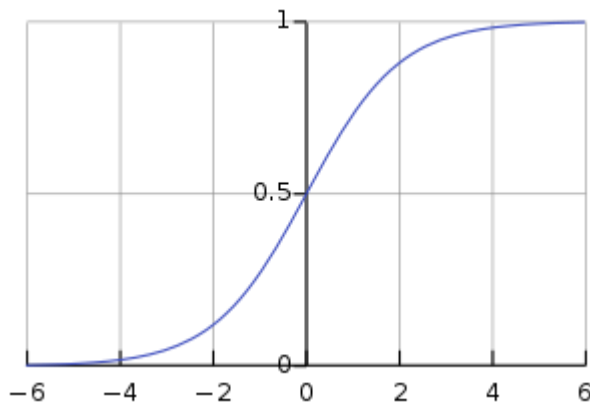
❖ Gradient vanishing problem

- Derivative of sigmoid and tanh is larger than 0 and smaller than 1.

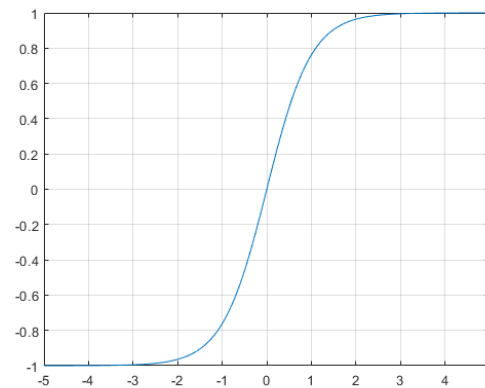
$$\text{sigmoid} = \frac{1}{1+e^{-x}}, \quad \tanh(x) = \frac{1-e^{-x}}{1+e^{-x}}$$

- BPTT unfolds RNNs into a very deep network
- Repeated multiplications $\rightarrow$ vanishing gradients

$\rightarrow$ Partial Sol: Long Short Term Memory (LSTM) or Gated Recurrent Unit (GRU) RNN



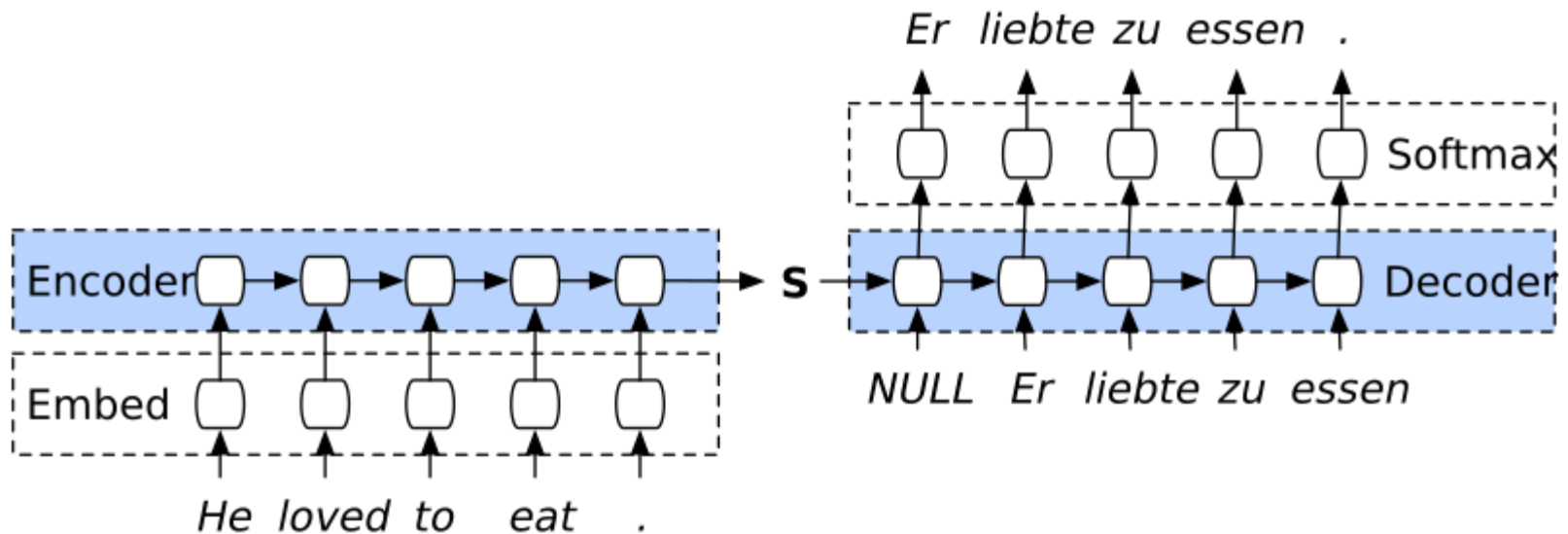
sigmoid



tanh

Recurrent Neural Network(RNN)

Machine Translation Example



Attention Example

❖ Sentence Example:

"The animal didn't cross the street because **it** was too tired."

- “Attention” confirms what **it** refers to.

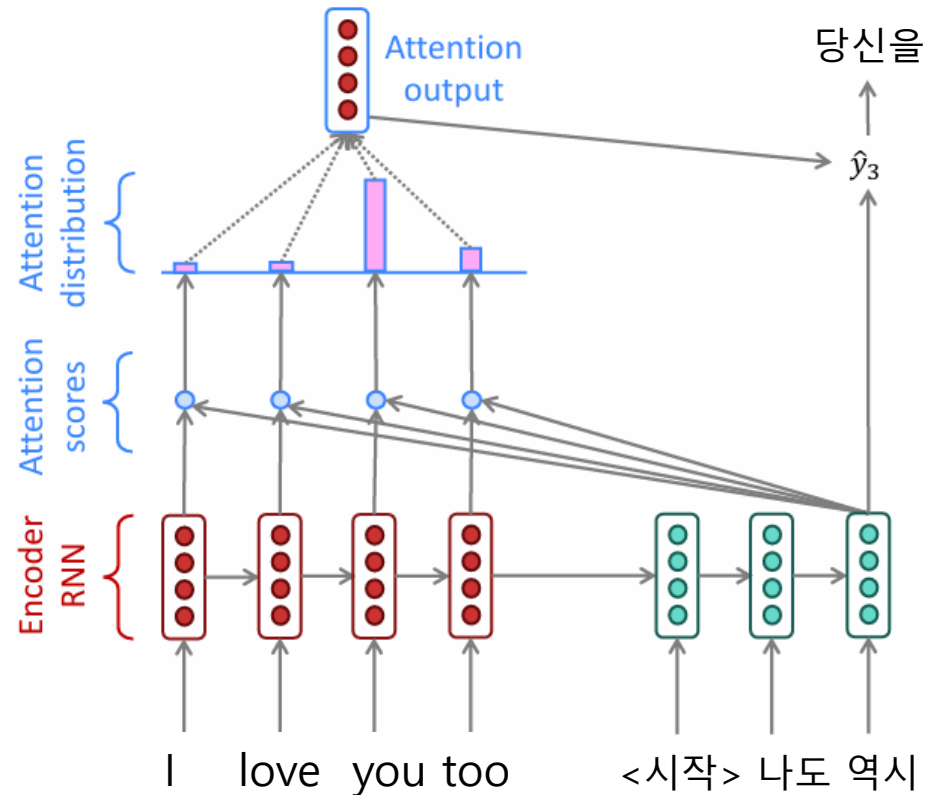
Attention in Machine Translation

❖ Attention

- Uses **all hidden states** instead of only the final state
- **Dot product** → **similarity (affinity)** for attention weights

❖ Benefits

- Captures **long-distance information**
- Reduces **information bottleneck**
- Still **sequential** → **slow computation**



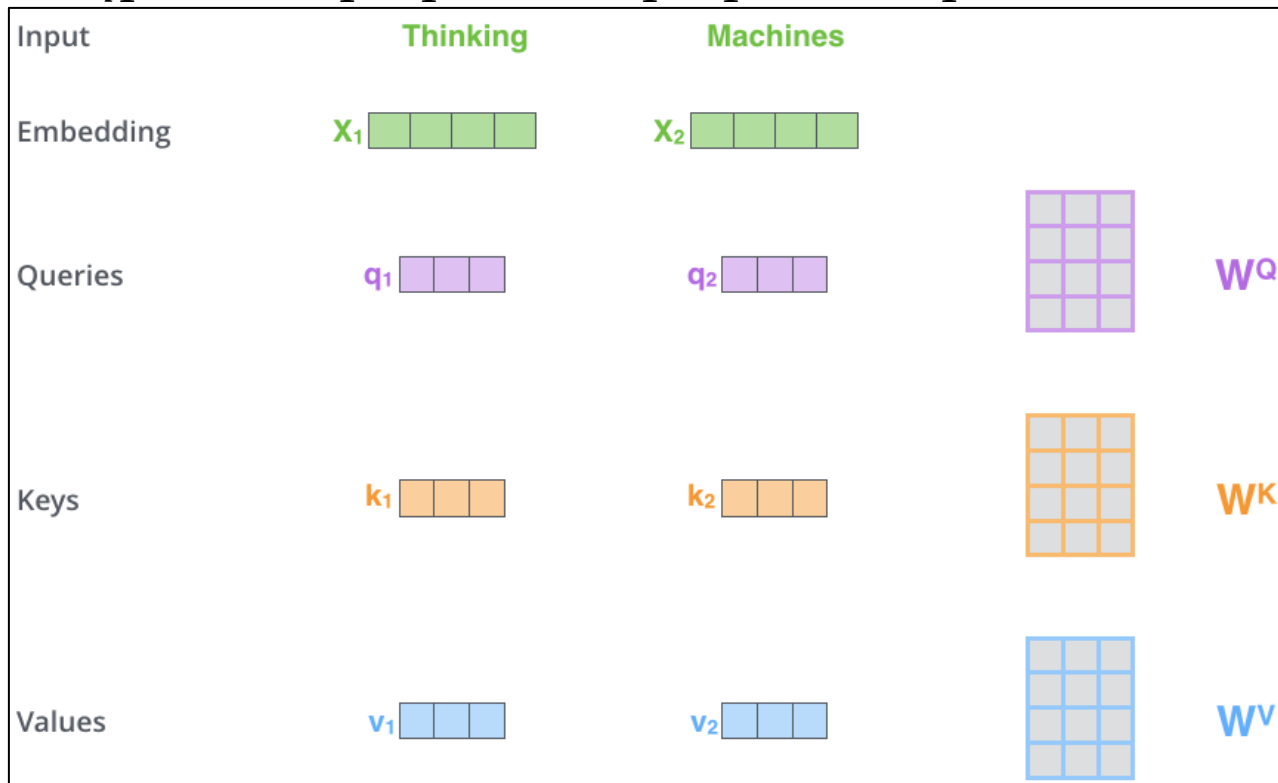
Self Attention Calculation

❖ Self Attention

- Attention to the word itself and the neighboring words in the same sentence
- Cf. Cross Attention: attention between source and target words in NMT

❖ Scale Dot Product Attention – Self Attention

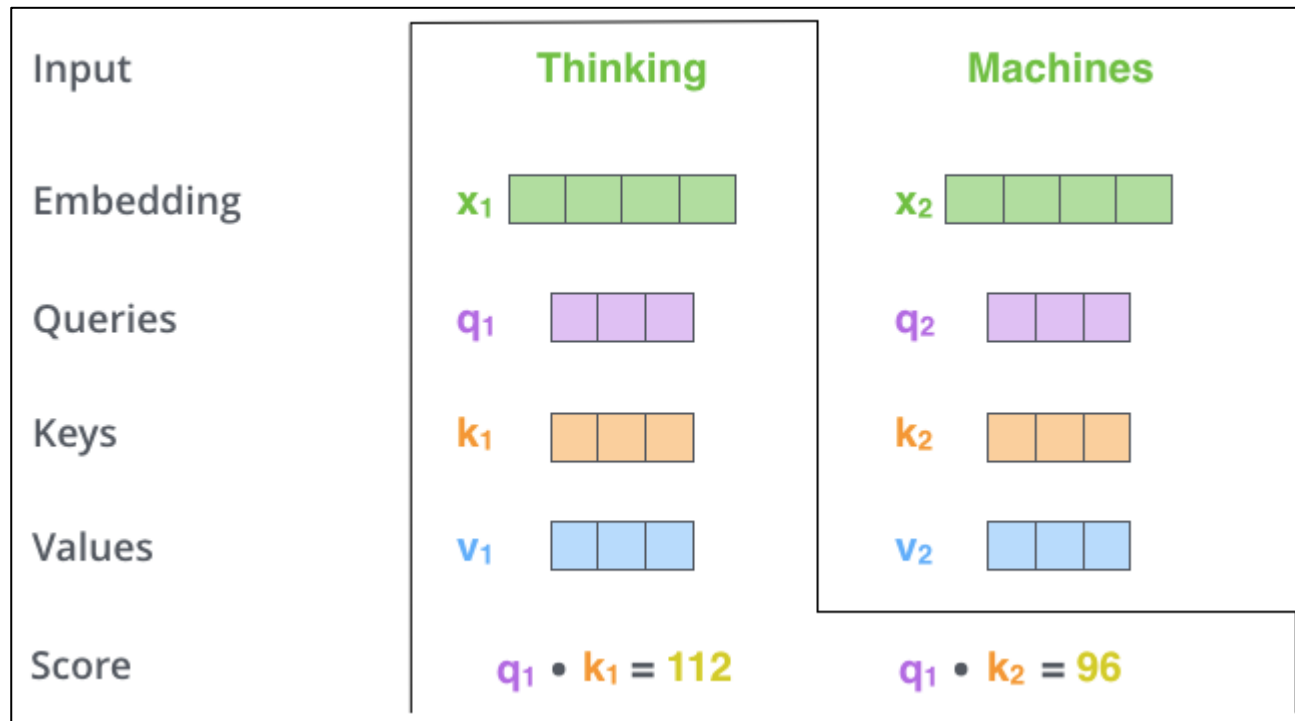
- Generate Query, Key, Value vectors using Trainable vectors (W^*)
- $q_1 = W^Q \cdot x_1$, $k_1 = W^K \cdot x_1$, $v_1 = W^V \cdot x_1$



Self Attention Calculation

❖ Scale Dot Product Attention – Self Attention

- Dot product of q and k (All combination of q and k - $O(n^2)$)
- $q_i \cdot k_j$ for $1 \leq i, j \leq n$



Self Attention Calculation

❖ Scale Dot Product Attention – Self Attention

▪ Scale

- To get stable gradient, divide the score by the square root of the size of Q^K
- ex) Divide score by 8 if $\text{sizeof}(Q^K) = 64$

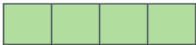
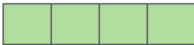
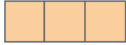
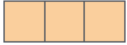
▪ Softmax

- Converts similarity to probability

$$\text{Softmax}(a_i) = \frac{e^{a_i}}{\sum_{j=1}^n e^{a_j}}$$

- ex:

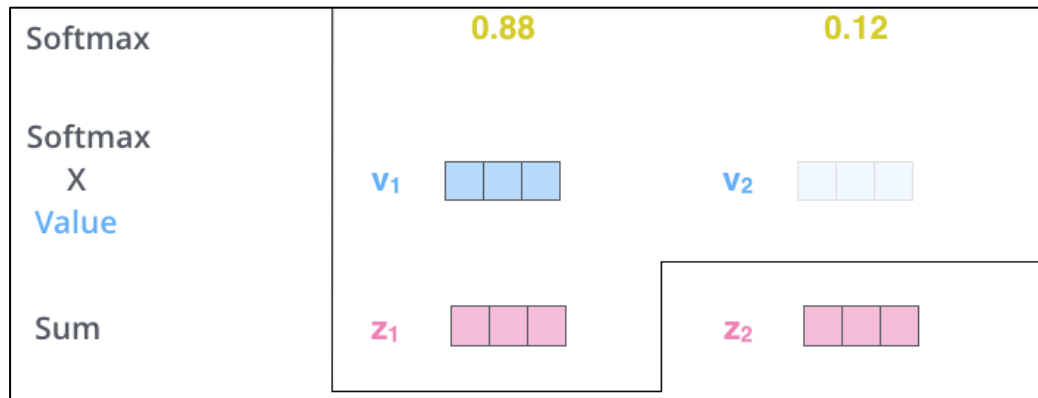
$$\frac{e^{14}}{e^{14} + e^{12}} = \frac{1.2M}{1.37M} = 0.88$$

Input	Thinking	Machines
Embedding	x_1 	x_2 
Queries	q_1 	q_2 
Keys	k_1 	k_2 
Values	v_1 	v_2 
Score	$q_1 \cdot k_1 = 112$	$q_1 \cdot k_2 = 96$
Divide by 8 ($\sqrt{d_k}$)	14	12
Softmax	0.88	0.12

Self Attention Calculation

❖ Scale Dot Product Attention – Self Attention

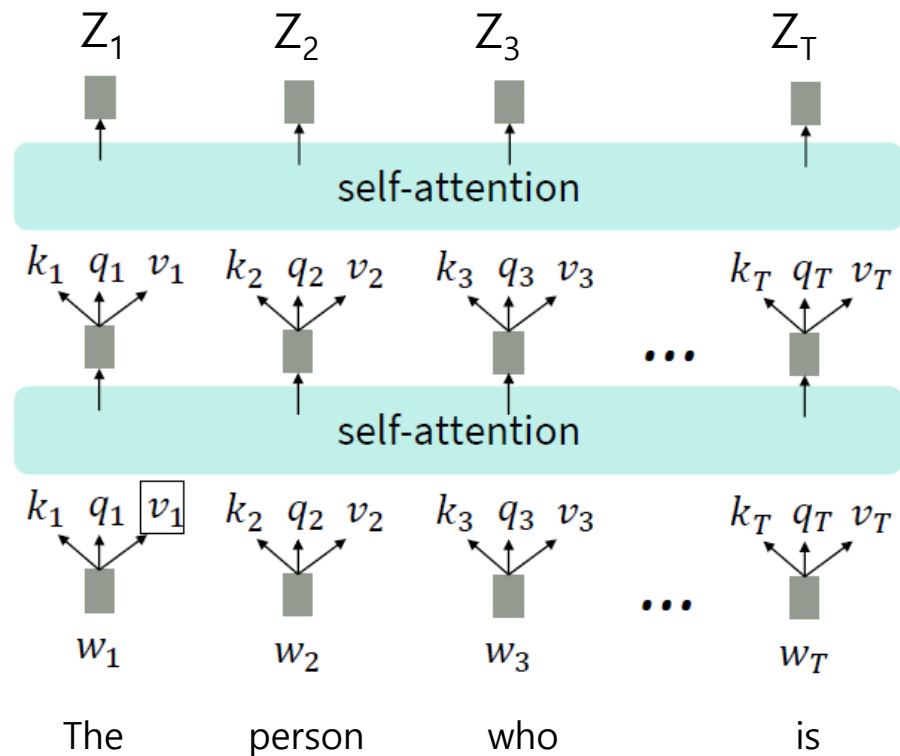
- Multiply each Value with the final Softmax value, and sum them all
- Example: new vector z_1 reflects v_1 itself and the other vector v_2



Transformer

❖ Base Form

- Stacking Self Attention
- Applying self attention repeatedly



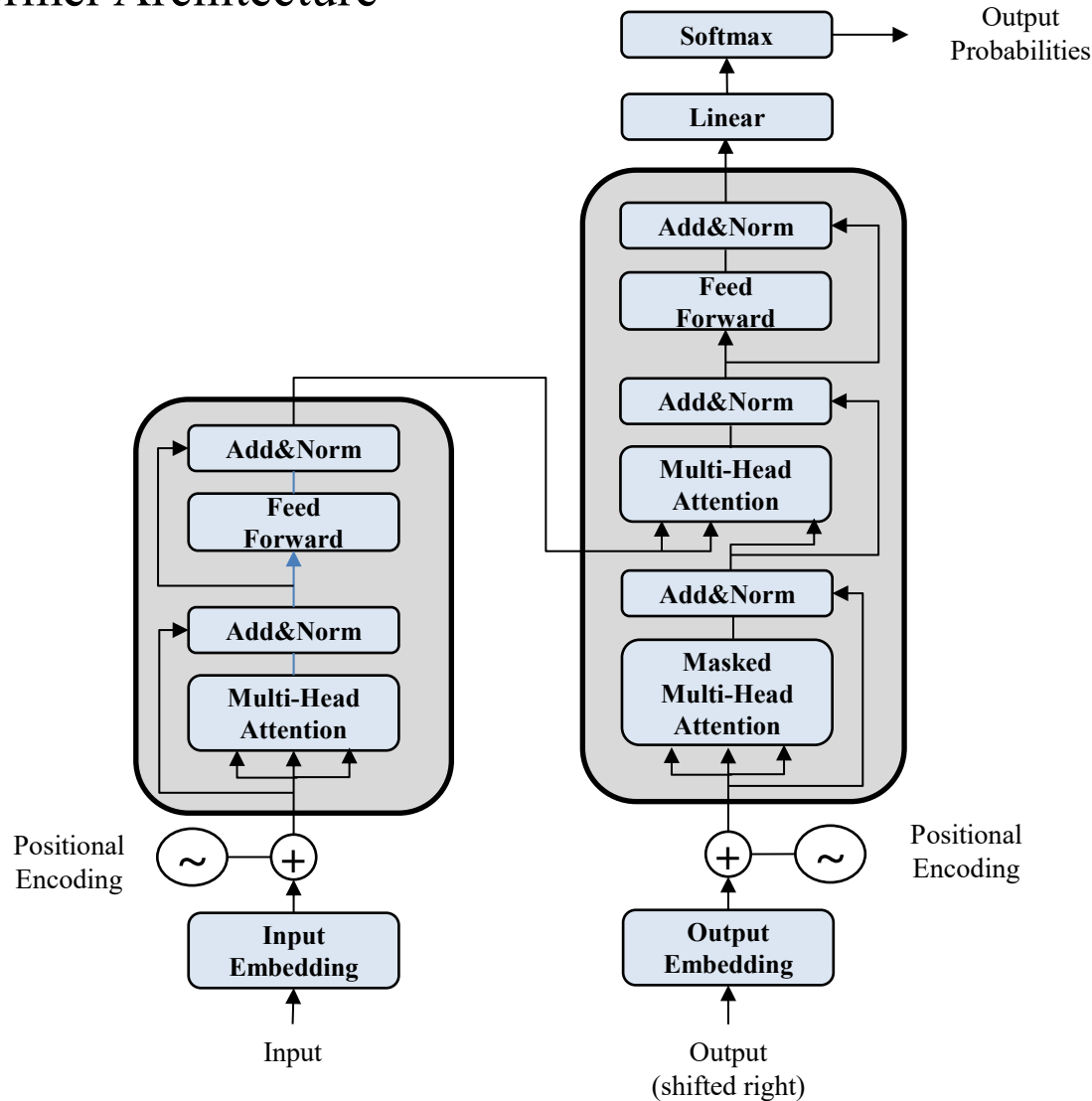
Transformer

❖ Improvement of Stacked Self Attention

1. Positional Encoding for word order
2. Feed Forward Layer of non-linearity
3. Masking for future words generation (for decoder training)
4. Multi-headed attention
5. Residual connection
6. Layer normalization
7. Scaled dot product

Transformer

❖ Transformer Architecture



Pre-Training and Fine Tuning

❖ Pre-training

- Train on **large unlabeled text corpora**
- Use **self-supervised objectives**
 - Next-word prediction
 - Masked word prediction
- Learn **general language knowledge**

❖ Fine-tuning

- Adapt the model to **specific tasks**
- Train with **task-specific labeled data**
- Requires **relatively small datasets**
- Examples: POS tagging, WSD, sentiment analysis, QA, summarization

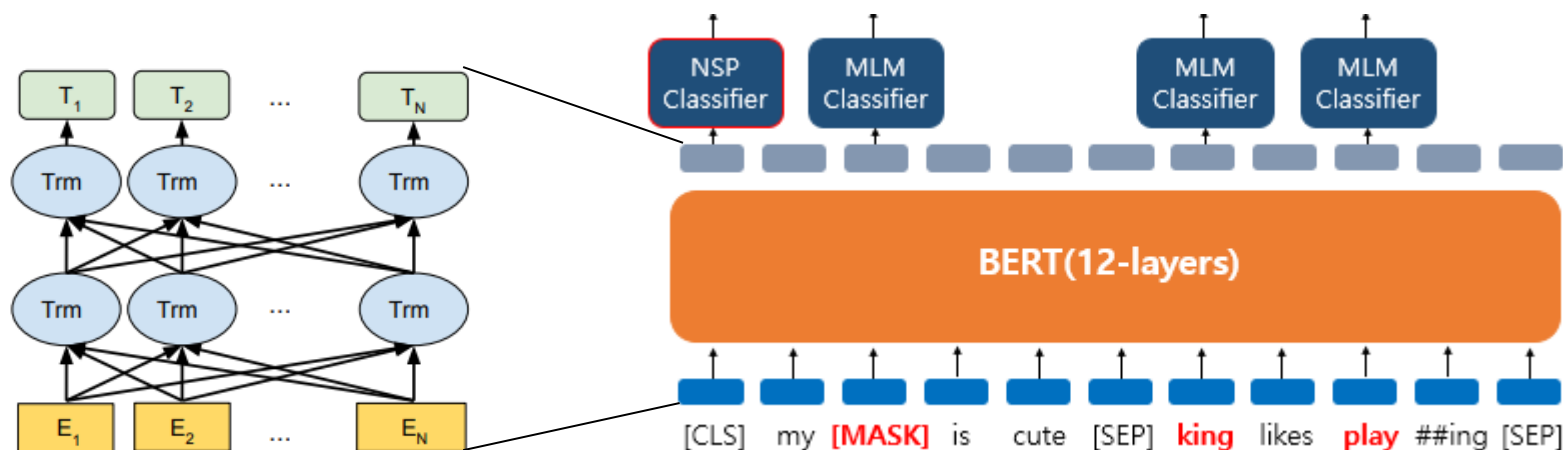
BERT

- ❖ BERT: Bi-directional Encoder Representations from Transformers
 - Trained transformer encoder
 - Bi-directional context
 - Pretraining with masked language modeling and next sentence prediction
- ❖ Subword token (Word piece)
 - Mitigate unknown word problem
 - playing -> play ##ing , retry -> re ##try
 - replaying -> re ##play ##ing (UNK: replay)
- ❖ Used for:
 - (Fine tuning tasks)
 - Sentence classification: SST-2, CoLA
 - Natural language inference: MNLI, RTE
 - Paraphrase detection: MRPC, QQP
 - Semantic similarity: STS-B
 - Question answering: SQuAD

BERT Pre-training

❖ Pre-training

- Masked language modeling
 - Change and restore 15% of words randomly selected
 - Change: 80% [mask] , 10% random word change, 10% same (no change)
- Next sentence prediction



BERT architecture

source: my **dog** is cute. **he** likes **playing**

BERT fine tuning

❖ BERT fine tuning tasks

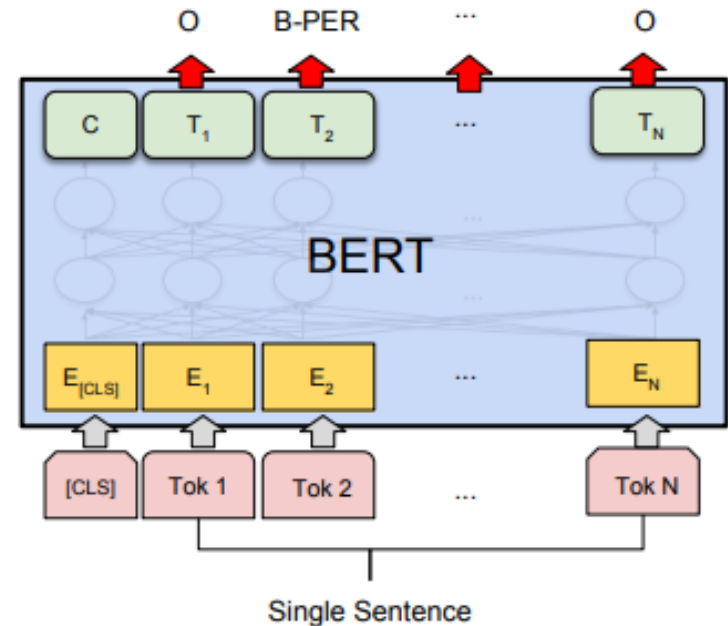
- Morphological Analysis (MA)
- Part-Of-Speech tagging
- Parsing
- Word Sense Disambiguation
- Semantic Role Labeling
- Named Entity Recognition
- Information Extraction (Relation Extraction)

❖ Classifier working example

- Assume 3 POS tags: N, V, A
- Inner product of output vector Z and POS vectors: N, V, A
- Calculate softmax of the results to choose the most probable POS

$$\text{POS} = \{N, V, A\}$$

$$\text{Argmax}_{\text{pos}}(\text{softmax}(Z \cdot \text{POS}))$$



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

GPT

❖ Generative Pretrained Transformer

- Transformer decoder
- Autoregressive language model
- Generates text token by token

❖ Pre-training

- Next word prediction

❖ Fine Tuning Tasks

- Same as BERT
- Solve the problems with the generation method

GPT

❖ Large Language Models

- GPT-4
- LLaMA
- Claude
- PaLM
- (cf: BERT is called as pre-trained LM or encoder model)

❖ Applications

- Chatbot
- Coding
- Text summarization
- Text generation(Writing)

Summary

- ❖ Perceptron
 - Computational model for biological neuron
 - Learning data without manual feature extraction
 - Multi-layer perceptron for complex pattern learning
- ❖ Recurrent Neural Network (RNN)
 - Recurrent architecture for sequential data
 - Error backpropagation through time
 - Vanishing gradient problem
 - Improved version: LSTM, GRU
- ❖ Transformer
 - Self attention
 - Attention using Query, Key, Value calculation
 - Encoder, Decoder
- ❖ Language model
 - BERT (encoder model)
 - GPT (decoder model, LLM)
 - pre-training and fine tuning